2026-11-01 11:10:11 You

ok go ahead

ChatGPT gpt-6-mini

Responsibility 1: Ansible Playbooks, Roles and Automation

Ansible – Advanced fundamentals

Your resume statement:

"Experienced in developing, structuring and managing reusable Ansible playbooks and roles, using tasks, handlers, variables, Jinja2 templates and conditional execution to standardize automation workflows."

This responsibility is about knowing how to structure Ansible automation professionally rather than writing a few commands in a single YAML file.

1. Simple technical definition

Ansible is an agentless automation and configuration management tool. It connects to remote Linux servers through SSH and executes predefined tasks using YAML-based playbooks.

There are two important concepts:

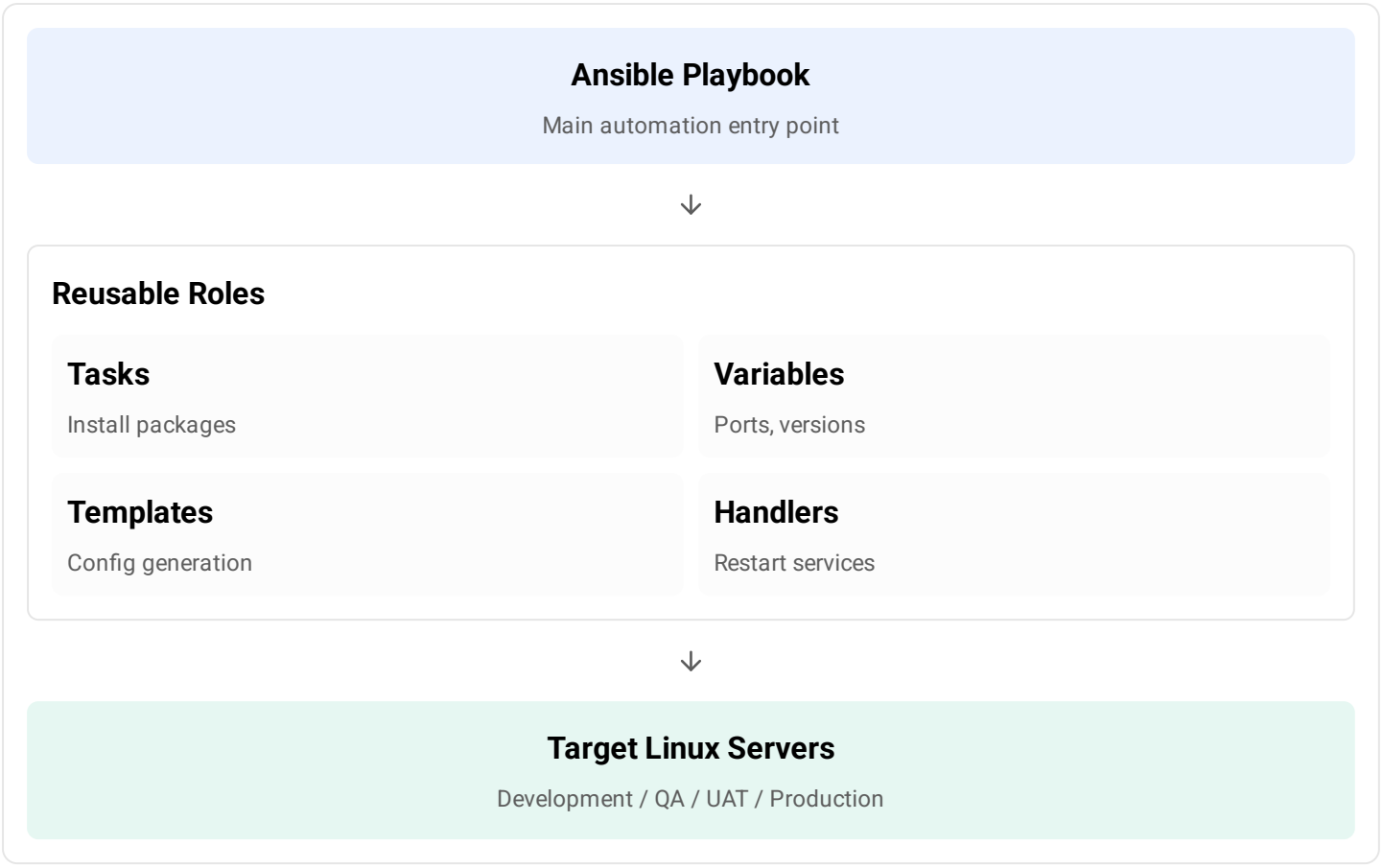
- **Playbook:** A YAML file containing instructions that define what automation should happen, on which servers and in what order.
- **Role:** A reusable, organized collection of tasks, variables, templates, handlers and other files that perform a particular responsibility.

Let's understand the individual terms in your resume.

Component	Simple explanation	Example
Tasks	Individual operations	Install Nginx
Handlers	Actions triggered by changes	Restart Nginx
Variables	Dynamic values	Port = 8080
Jinja2	Dynamic template engine	Generate configuration files
Conditions	Execute tasks based on conditions	Install package only on RHEL
Roles	Reusable automation structure	Web server setup

Playbook	Overall execution instructions	Configure all web servers

How they connect



2. Storytelling – Project explanation

Imagine your project has multiple application servers running on AWS EC2.

Initially, whenever a new server was introduced, the team had to manually install packages, create application users, configure services and update configuration files.

As the number of servers increased, maintaining the same configuration everywhere became difficult.

We structured our Ansible automation into reusable roles.

For example, we created separate roles for:

- Web server configuration.
- Application dependency installation.
- Monitoring agent installation.
- User and permission management.
- Service configuration.

Each role contained its own tasks, variables, templates and handlers.

For example, whenever we needed to configure a new application server, instead of writing everything again, we could call the existing application setup role from a main playbook.

We also used variables for environment-specific configurations and Jinja2 templates to generate configuration files dynamically.

This helped us reuse automation across environments and maintain consistent server configurations.

The main idea to remember: Instead of writing automation repeatedly, we built reusable components and combined them whenever required.

3. Real-world task with an incident

Task: Automating Nginx installation and configuration across multiple application servers.

Imagine we have three application servers and need to install and configure Nginx on all of them.

Instead of manually connecting to each server, we create an Ansible role.

Step 1: Create the role structure

```
</> Bash

ansible-galaxy init nginx_setup
```

This generates:

```
</> Plain text

nginx_setup/
├── defaults/
│   └── main.yml
├── handlers/
│   └── main.yml
├── tasks/
│   └── main.yml
├── templates/
│   └── nginx.conf.j2
├── vars/
│   └── main.yml
└── files/
```

Step 2: Define variables

`defaults/main.yml`

```yaml
nginx_port: 8080
nginx_worker_processes: auto
```

Variables allow us to change configuration without modifying the actual tasks.

For example, QA might use port 8080 while production uses port 80.

Step 3: Create tasks

`tasks/main.yml`

```yaml
- name: Install Nginx
  package:
    name: nginx
    state: present

- name: Configure Nginx
  template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
  notify: Restart Nginx

- name: Start Nginx
  service:
    name: nginx
    state: started
    enabled: yes
```

Notice `notify: Restart Nginx`.

This triggers a handler only when the configuration task reports a change.

Step 4: Create Jinja2 template

`templates/nginx.conf.j2`

```nginx
```

```
worker_processes {{ nginx_worker_processes }};

events {
    worker_connections 1024;
}

http {
    server {
        listen {{ nginx_port }};
    }
}
```

Jinja2 dynamically replaces the variables with their actual values.

Step 5: Create handler

`handlers/main.yml`

```yaml
- name: Restart Nginx
  service:
    name: nginx
    state: restarted
```

The handler restarts Nginx when its configuration changes.

Step 6: Execute the role using a playbook

`site.yml`

```yaml
- name: Configure web servers
  hosts: web_servers
  become: yes

  roles:
    - nginx_setup
```

Execution:

```bash
ansible-playbook -i inventory site.yml
```

Incident: Nginx configuration update causes service failure

Imagine we update the Nginx port from 8080 to 9090, but the service fails to restart because the generated configuration contains an invalid directive.

How would we troubleshoot?

1. Check Ansible execution output.
2. Connect to the affected server.
3. Validate the Nginx configuration.
4. Check the systemd service status.
5. Inspect Nginx error logs.
6. Correct the Jinja2 template or variable.
7. Re-run the playbook.
8. Verify service availability.

Commands:

```bash
nginx -t

systemctl status nginx

journalctl -u nginx -xe

tail -f /var/log/nginx/error.log
```

This is a good example of how tasks, templates, handlers and variables work together.

4. Common errors and issues

Issue	Possible reason	Troubleshooting
Role not found	Incorrect directory structure or roles path	Verify role location
YAML syntax error	Incorrect indentation	Run syntax check
Handler not executed	Task didn't report a change or notify name mismatch	Check `notify` and handler

Undefined variable	Variable not declared or incorrect scope	Verify variable names and precedence
Template rendering failure	Invalid Jinja2 syntax	Check `.j2` template
Task skipped unexpectedly	Incorrect condition	Validate `when` expression
Permission denied	Insufficient privileges	Use `become: yes`
Playbook runs repeatedly with changes	Non-idempotent task	Use appropriate Ansible modules

Useful commands:

```bash
ansible-playbook site.yml --syntax-check

ansible-playbook site.yml --check

ansible-playbook site.yml --list-tasks

ansible-playbook site.yml -vvv
```

5. Client interview cross-questions

These are the questions you should be able to answer after understanding this responsibility.

1. What is the difference between an Ansible playbook and a role?
2. What is the difference between tasks and handlers?
3. When does an Ansible handler execute?
4. What is Jinja2 and where have you used it?
5. What is variable precedence in Ansible?
6. What is the purpose of the `when` condition?
7. What is idempotency?
8. What is the difference between `include_tasks` and `import_tasks`?
9. How do you debug a failed Ansible playbook?
10. How do you organize roles for multiple environments?

Responsibility 2: Linux Server Automation Using Ansible

Ansible + Linux Administration

Your resume statement:

"Solid knowledge of Linux server automation using Ansible, including OS-level configurations, package and dependency management, user administration, filesystem permissions, systemd service management and security configurations."

This responsibility is about automating everyday Linux system administration activities across multiple servers.

1. Simple technical definition

Linux server automation using Ansible means managing and configuring Linux servers remotely without manually logging into every machine.

Ansible connects through SSH and uses Linux-related modules to perform administrative operations.

Let's break down your resume statement.

Area	What it means	Example
OS-level configuration	Configure operating system settings	Hostname, timezone, kernel parameters
Package management	Install, update and remove software	Install Java, Nginx
Dependency management	Maintain required software dependencies	Java runtime, libraries
User administration	Create and manage Linux users	Create `appuser`
Filesystem permissions	Control file access	Owner, group, 755 permissions
Systemd management	Manage background services	Start Jenkins, Nginx
Security configuration	Apply server security settings	SSH restrictions, firewall rules

Important: RHEL uses DNF/YUM and Ubuntu uses APT for package management. Ansible's generic `package` module can abstract some of these differences.

2. Storytelling – Project explanation

Imagine our application runs on multiple Linux servers across development, QA and production environments.

Each server needs certain configurations before it can support application deployment.

For example, application servers require Java, monitoring agents, dedicated application users, appropriate directory permissions and running services.

Instead of manually performing these activities on every server, we used Ansible playbooks to automate Linux administration.

We maintained common configurations through reusable tasks and variables. For example, we automated package installation, user creation, directory permissions and systemd service management.

Whenever a new server was introduced or an existing server needed configuration updates, the corresponding playbook could be executed against the required inventory group.

This helped maintain consistent configurations and reduced repetitive manual administration.

Remember this sequence: Prepare OS → Install packages → Create users → Configure permissions → Manage services → Apply security.

3. Real-world task with an incident

Task: Preparing a RHEL application server for deployment.

Imagine a new RHEL server has been provisioned in AWS. Before deploying the application, we need to configure it.

Step 1: Install required packages

RHEL example:

```yaml
- name: Install application dependencies
  hosts: app_servers
  become: yes

  tasks:
    - name: Install Java and required utilities
      dnf:
        name:
          - java-17-openjdk
          - curl
          - wget
          - unzip
        state: present
```

This ensures the required packages are installed.

Step 2: Create application user

We shouldn't generally run enterprise applications directly as root.

```yaml
- name: Create application user
  user:
    name: appuser
    shell: /bin/bash
    create_home: yes
    state: present
```

This creates a dedicated user for application operations.

Step 3: Configure directories and permissions

```yaml
- name: Create application directory
  file:
    path: /opt/myapp
    state: directory
    owner: appuser
    group: appuser
    mode: '0755'
```

Here:

- Owner: `appuser`
- Group: `appuser`
- Permissions: `755`

The application user gets full directory permissions, while other users get read and execute permissions.

Step 4: Configure systemd service

Suppose we have a Java application running as a JAR file.

We can create a systemd service file using Ansible.

```ini
[Unit]
Description=My Application
After=network.target
```

```
[Service]
User=appuser
WorkingDirectory=/opt/myapp
ExecStart=/usr/bin/java -jar /opt/myapp/app.jar
Restart=always

[Install]
WantedBy=multi-user.target
```

Ansible task:

```yaml
- name: Start application service
  systemd:
    name: myapp
    enabled: yes
    state: started
    daemon_reload: yes
```

This configures the service to start automatically when the server boots.

Step 5: Apply security configurations

Some common tasks include:

- Restricting unnecessary SSH access.
- Ensuring firewall rules are configured.
- Managing sudo permissions.
- Applying OS security updates.
- Disabling unnecessary services.

For example:

```yaml
- name: Ensure firewalld is running
  service:
    name: firewalld
    state: started
    enabled: yes
```

In a real environment, firewall changes must be carefully validated so they don't accidentally block SSH or application traffic.

Incident: Application service fails after server reboot

Imagine the application was working properly, but after a server reboot, users cannot access it.

What could have happened?

- Service wasn't enabled at boot.
- Application user doesn't have permission to access the JAR.
- Java dependency is missing.
- Incorrect systemd configuration.
- Application starts before a required dependency is available.

Troubleshooting:

1. Check service status

```bash
systemctl status myapp
```

2. Check application logs

```bash
journalctl -u myapp -xe
```

3. Verify Java installation

```bash
java -version
```

4. Check permissions

```bash
ls -l /opt/myapp/
```

5. Verify systemd configuration

```bash
systemctl cat myapp
```

After identifying the issue, correct the configuration using Ansible and validate the service.

4. Common errors and issues

Issue	Possible reason	Troubleshooting
Package installation failure	Repository unavailable	Check DNF/APT repositories
User creation failure	Existing user or invalid parameters	Check `/etc/passwd`
Permission denied	Incorrect ownership or permissions	Use `ls -l`, `chown`, `chmod`
Service failed to start	Invalid service configuration	Check `systemctl status`
Service not starting after reboot	Service not enabled	Check `systemctl is-enabled`
Java application fails	Incorrect Java version	Check `java -version`
SSH connectivity failure	SSH configuration or firewall issue	Check SSH service and security rules
Disk full	Logs or temporary files consuming space	Check `df -h`, `du -sh`
High CPU utilization	Process consuming excessive CPU	Check `top`, `ps`
Security update failure	Repository or package dependency conflict	Check package manager output

Important RHEL commands

Since RHEL is specifically mentioned in your resume, understand these commands well.

Purpose	Command
OS version	`cat /etc/redhat-release`
Package installation	`dnf install nginx`
Package update	`dnf update`
Check service	`systemctl status nginx`
Start service	`systemctl start nginx`
Enable at boot	`systemctl enable nginx`
Firewall status	`firewall-cmd --state`
SELinux status	`getenforce`

Disk utilization	`df -h`
Memory utilization	`free -h`
Process monitoring	`top`
System logs	`journalctl -xe`

5. Client interview cross-questions

1. How does Ansible connect to Linux servers?
2. What is the difference between `yum`, `dnf` and `apt`?
3. How do you create users and groups using Ansible?
4. What is the difference between `chmod` and `chown`?
5. What is systemd?
6. Difference between `systemctl start` and `systemctl enable`?
7. How do you troubleshoot a failed Linux service?
8. What is SELinux in RHEL?
9. Difference between permissive and enforcing modes in SELinux?
10. How do you perform patch management on Linux servers?
11. How do you check CPU, memory and disk utilization?
12. How do you ensure Ansible doesn't unnecessarily modify existing configurations?

Quick memory revision

OS → Packages → Users → Permissions → Services → Security

If the interviewer asks what you automated using Ansible in Linux, connect your answer through these six activities.

Next: Responsibility 3 – AWS Infrastructure Automation Using Ansible, covering EC2 configuration, security groups, application environment setup and AWS dynamic inventory.